

GOTO **COPENHAGEN** 2025

September 29 - October 3 • TAP1, Copenhagen • gotocph.com

HOW DEBUGGERS WORK

GOTO COPENHAGEN ⁺ ○ •

Sy Brand

they/them

Staff Software Engineer, Web Assembly @ Fastly

@TartanLlama

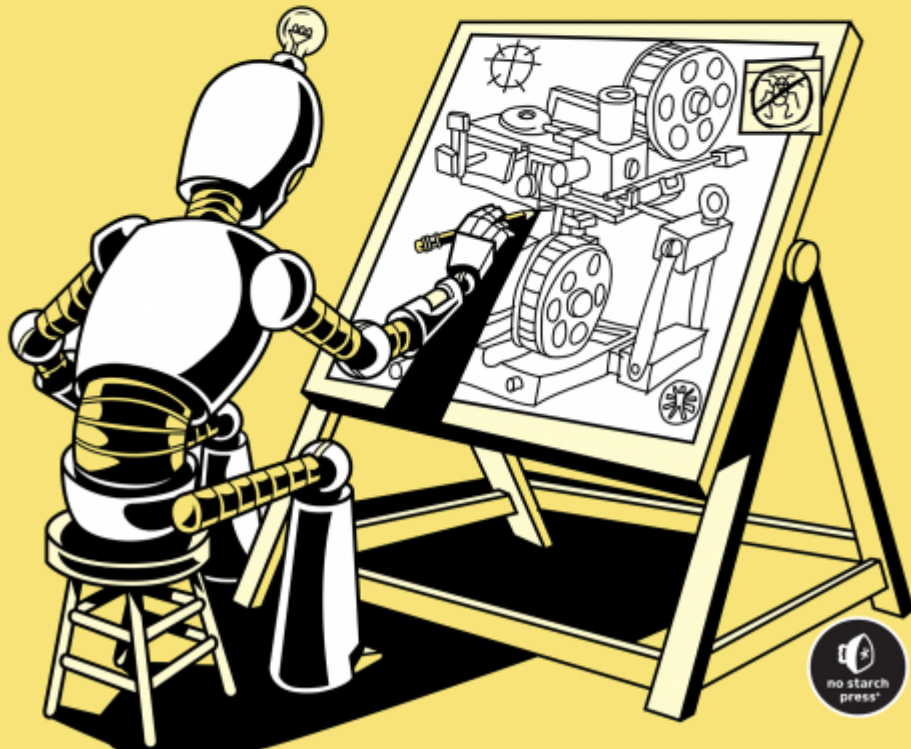
DOWNLOAD & USE **THE GOTO GUIDE APP**

DOWNLOAD THE APP
CREATE YOUR SCHEDULE
ASK QUESTIONS
RATE SESSIONS



BUILDING A DEBUGGER

SY BRAND



<https://nostarch.com/building-a-debugger>

What Kind of Debugger?

- Native code debuggers
 - For compiled languages
 - C, C++, Swift, Rust
- Examples:
 - IDE debugger (Visual Studio, Clion)
 - Command line debugger (GDB, LLDB)



AGENDA

Debug scenario architecture

Manipulating memory

Breakpoints

Watchpoints

Stepping

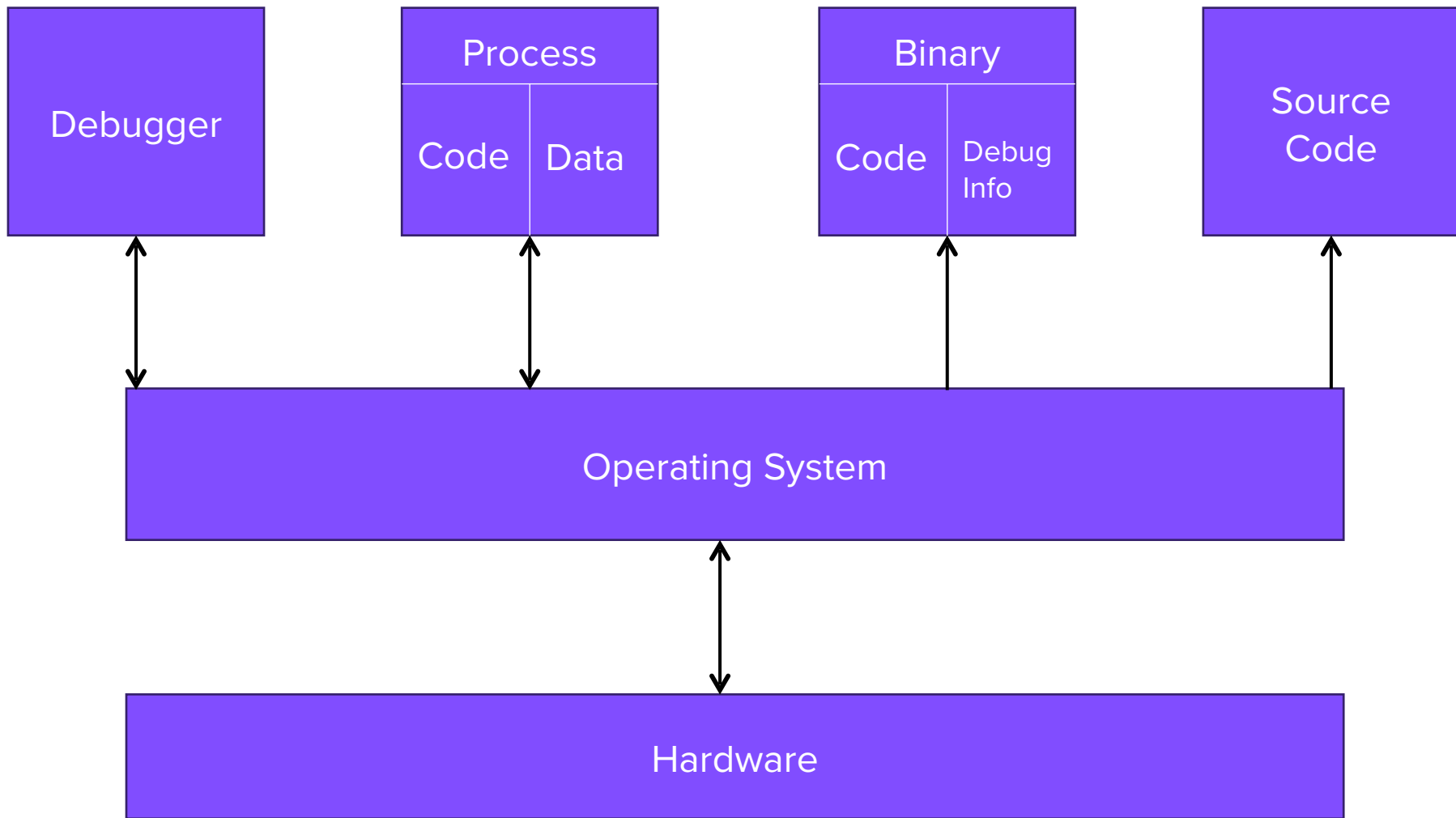
Stack unwinding

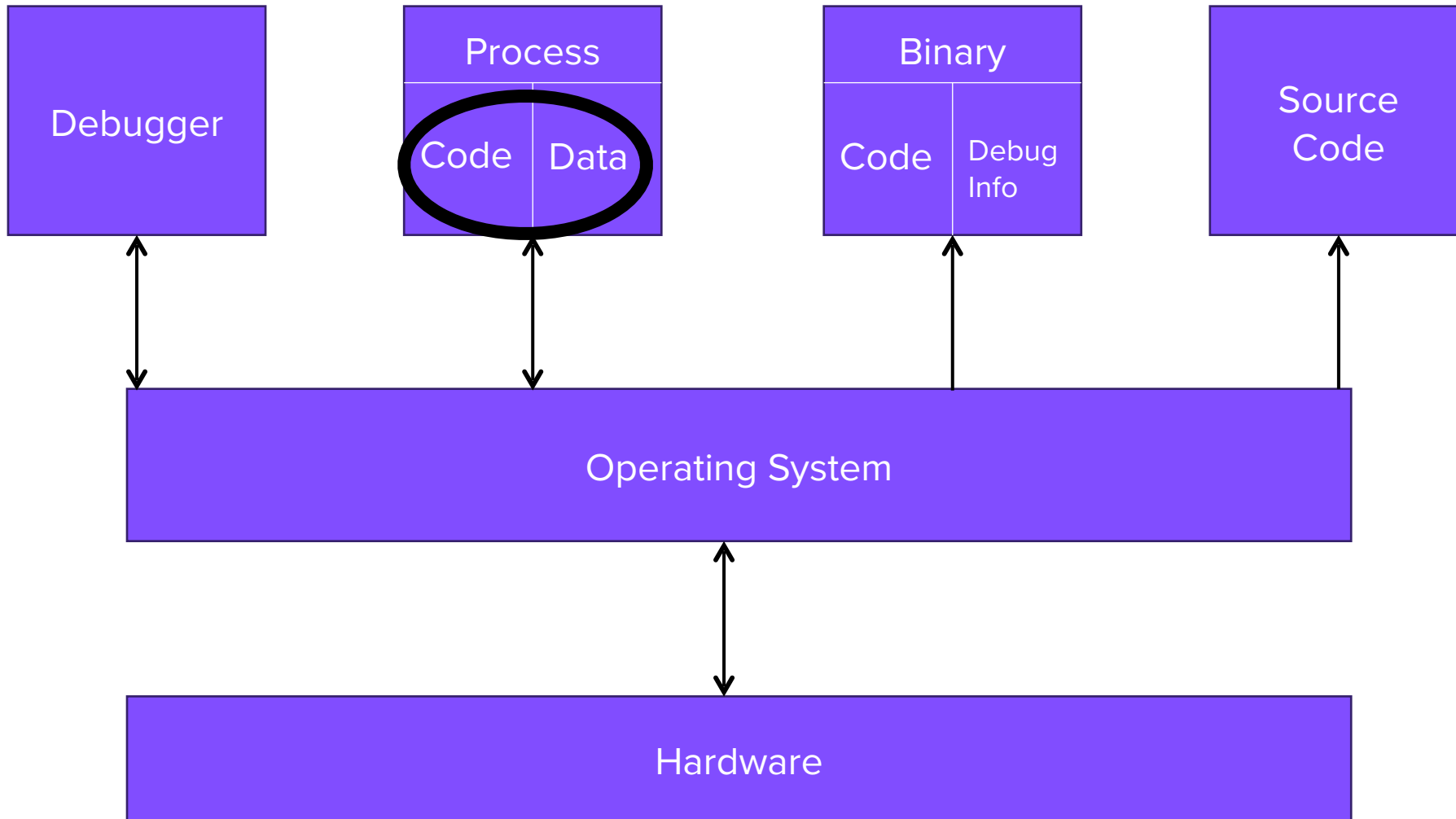
Reading a variable

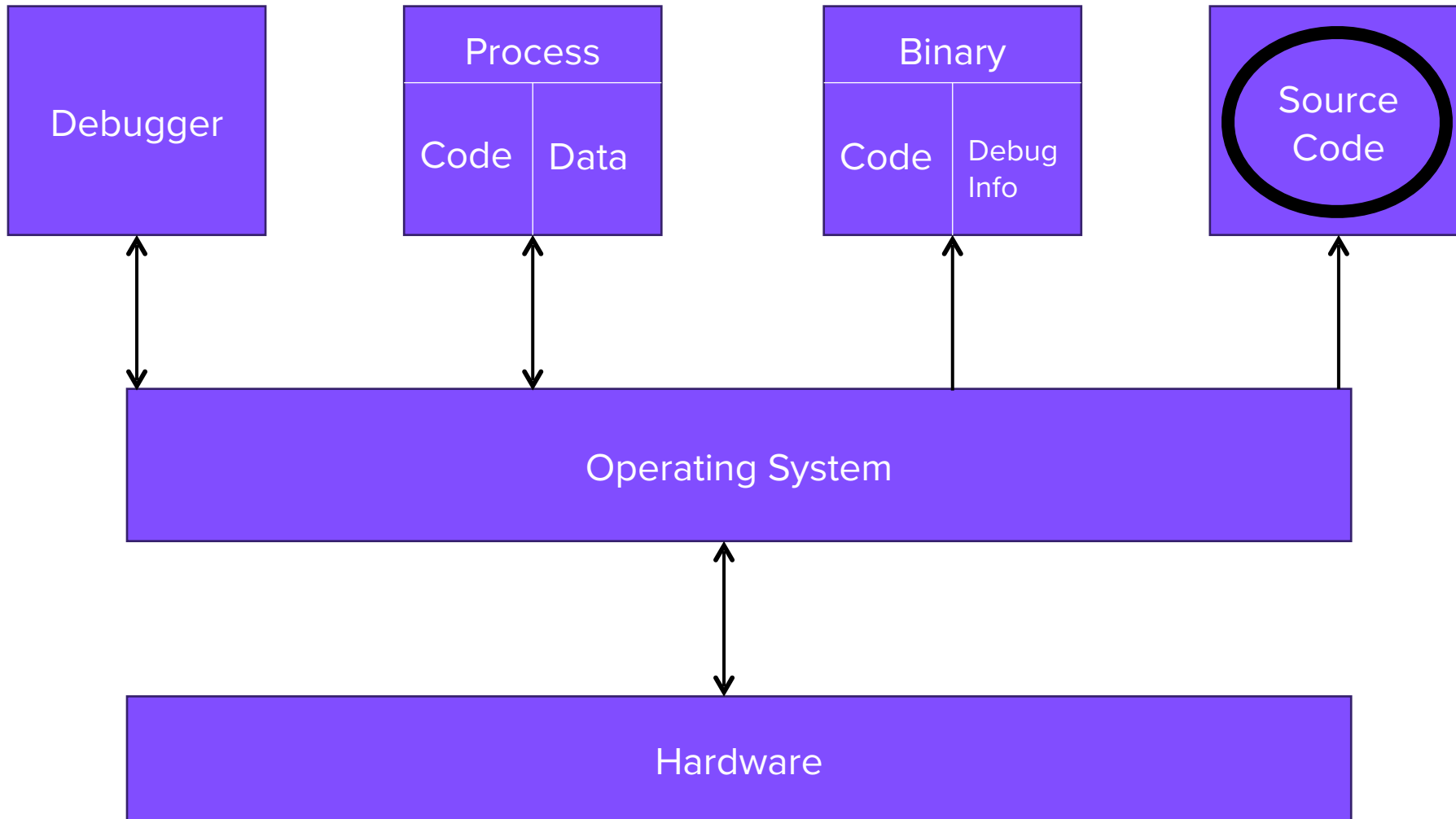
Calling a function



DEBUG SCENARIO ARCHITECTURE

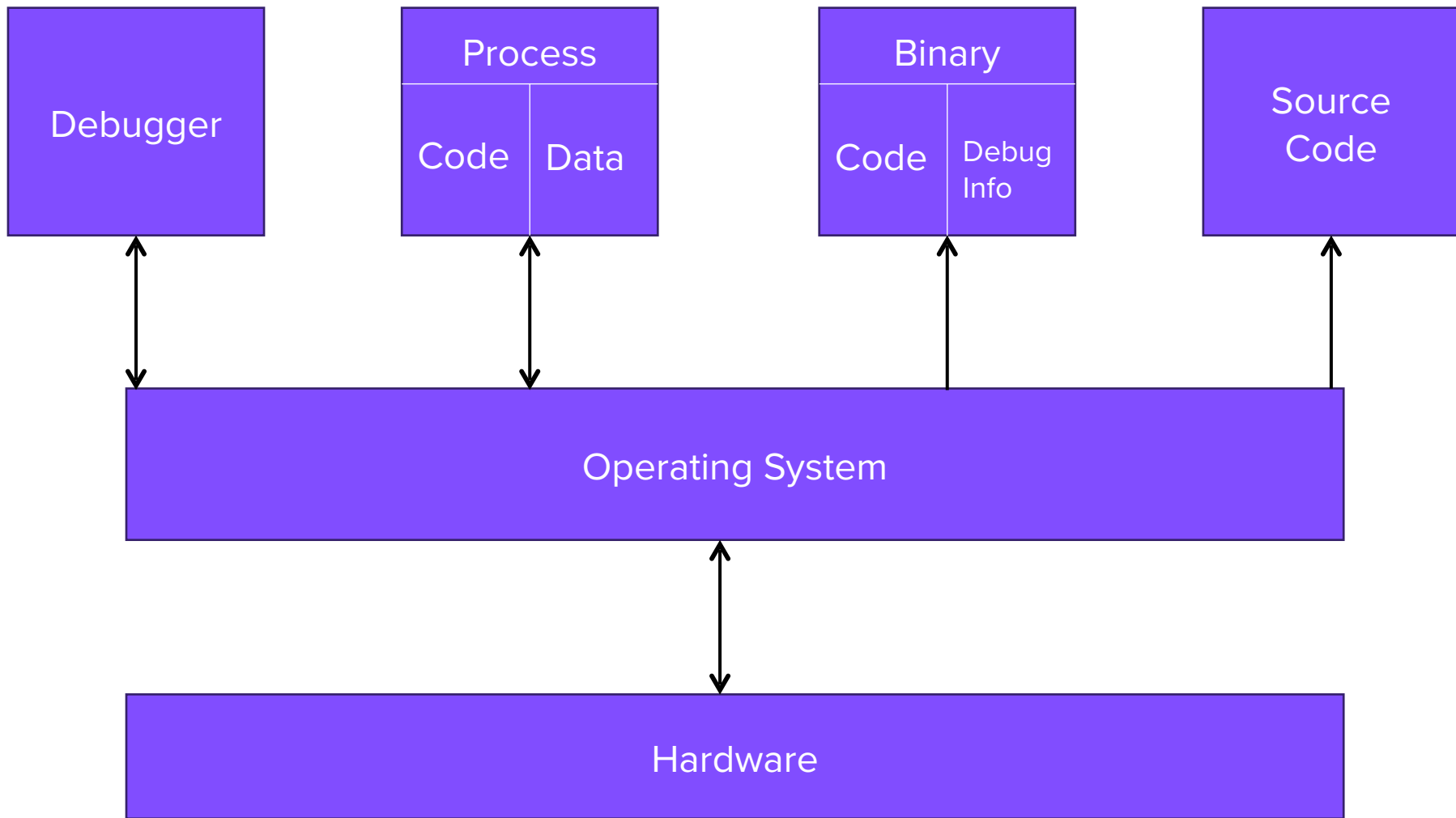


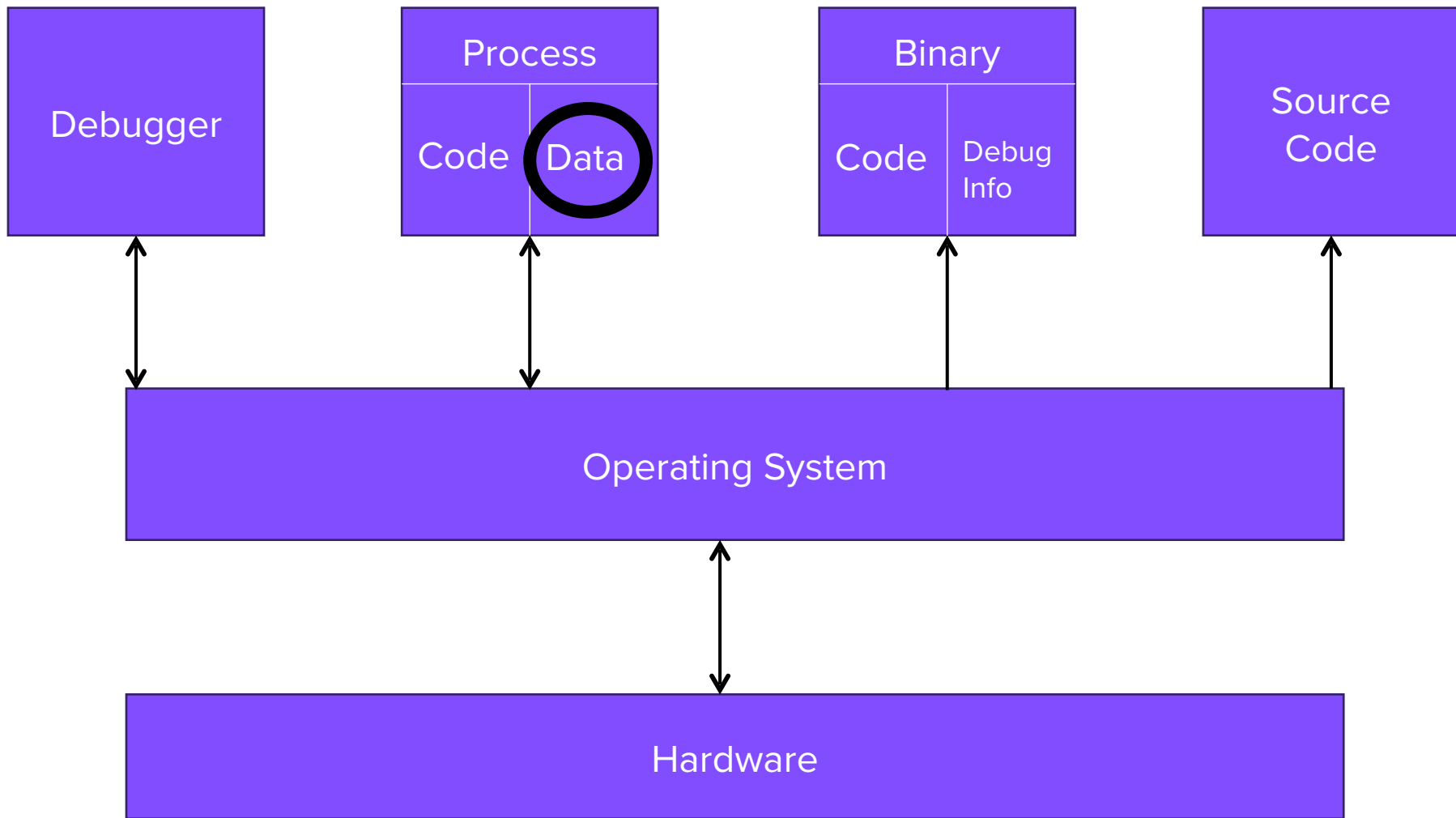


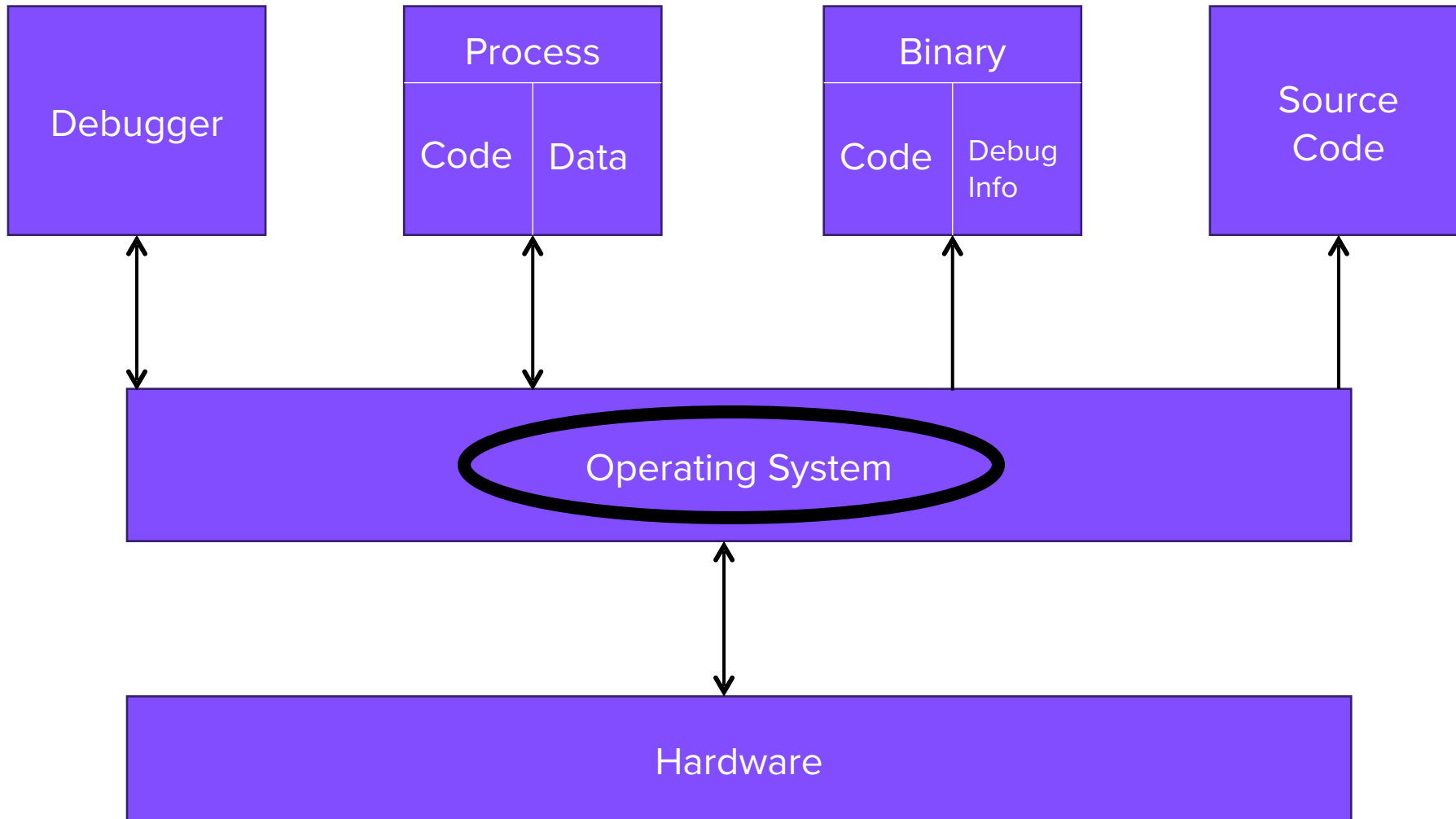




MANIPULATING MEMORY







SYSTEM CALLS (SYSCALLS)

Functions provided by the operating system to allow user programs to perform privileged operations

ptrace

```
uint64_t data = ptrace(  
    PTRACE_PEEKDATA,  
    pid,  
    address,  
    nullptr  
);
```

ptrace

- Some supported operations
 - Read and write memory/registers
 - Step over a single instruction
 - Continue a halted process
 - Attach to a running process

ptrace

- Some supported operations
 - Read and write memory/registers
 - Step over a single instruction
 - Continue a halted process
 - Attach to a running process

```
struct user_regs_struct {  
    unsigned long r15;    unsigned long r14;  
    unsigned long r13;    unsigned long r12;  
    unsigned long rbp;    unsigned long rbx;  
    unsigned long r11;    unsigned long r10;  
    unsigned long r9;     unsigned long r8;  
    unsigned long rax;    unsigned long rcx;  
    unsigned long rdx;    unsigned long rsi;  
    unsigned long rdi;    unsigned long orig_rax;  
    unsigned long rip;    unsigned long cs;  
    unsigned long eflags; unsigned long rsp;  
    unsigned long ss;     unsigned long fs_base;  
    unsigned long gs_base; unsigned long ds;  
    unsigned long es;     unsigned long fs;  
    unsigned long gs;  
};
```

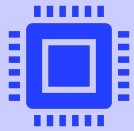
```
user_regs_struct regs;  
ptrace(PTRACE_GETREGS, pid, nullptr, &regs);
```

BREAKPOINTS



Set a breakpoint on line 149 of pet_cat.c

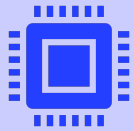




Which machine instruction corresponds to that source line?



How do we make the process stop when that machine instruction is executed?



Which machine instruction corresponds to that source line?



How do we make the process stop when that machine instruction is executed?

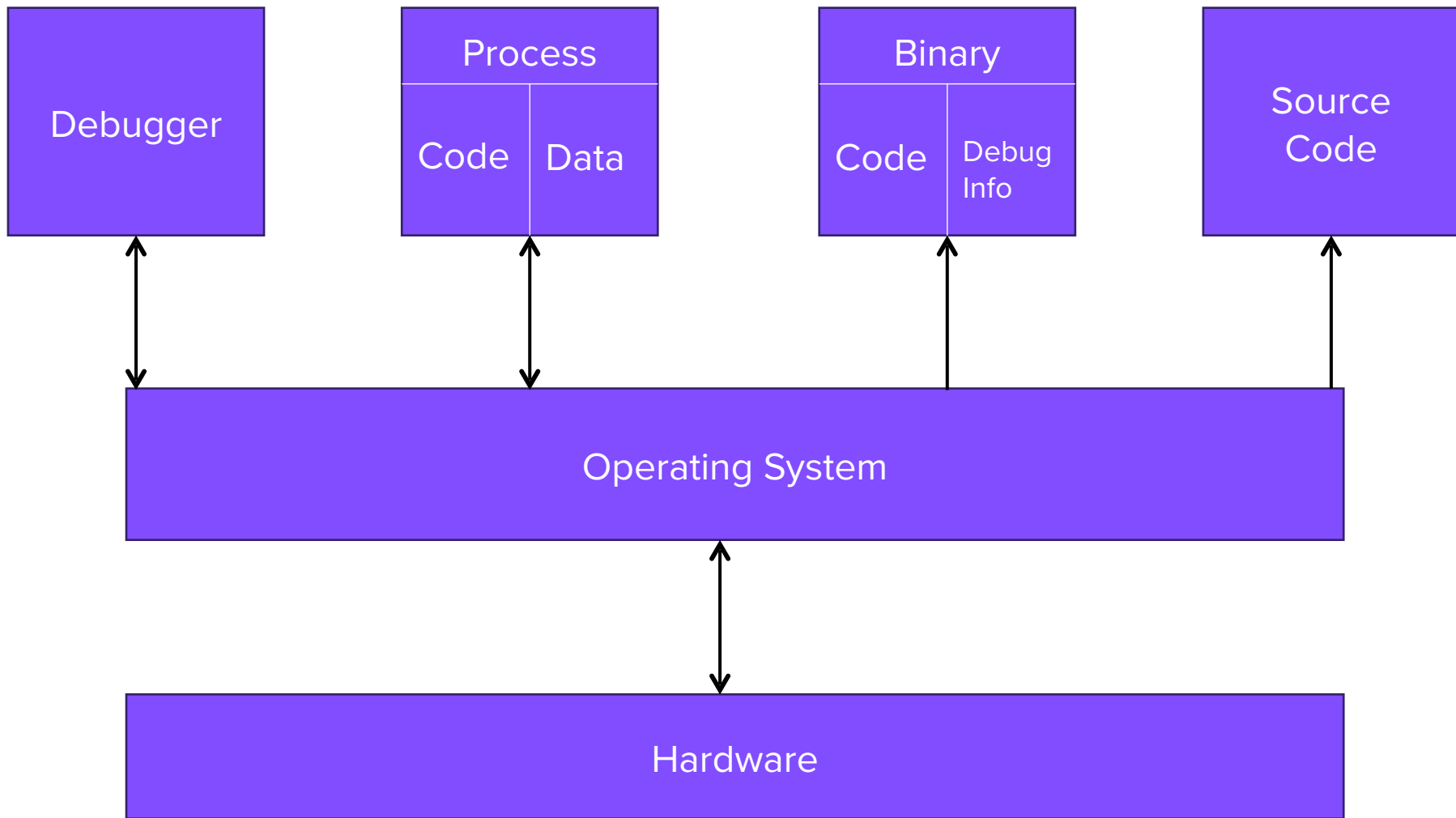
DEBUG INFORMATION

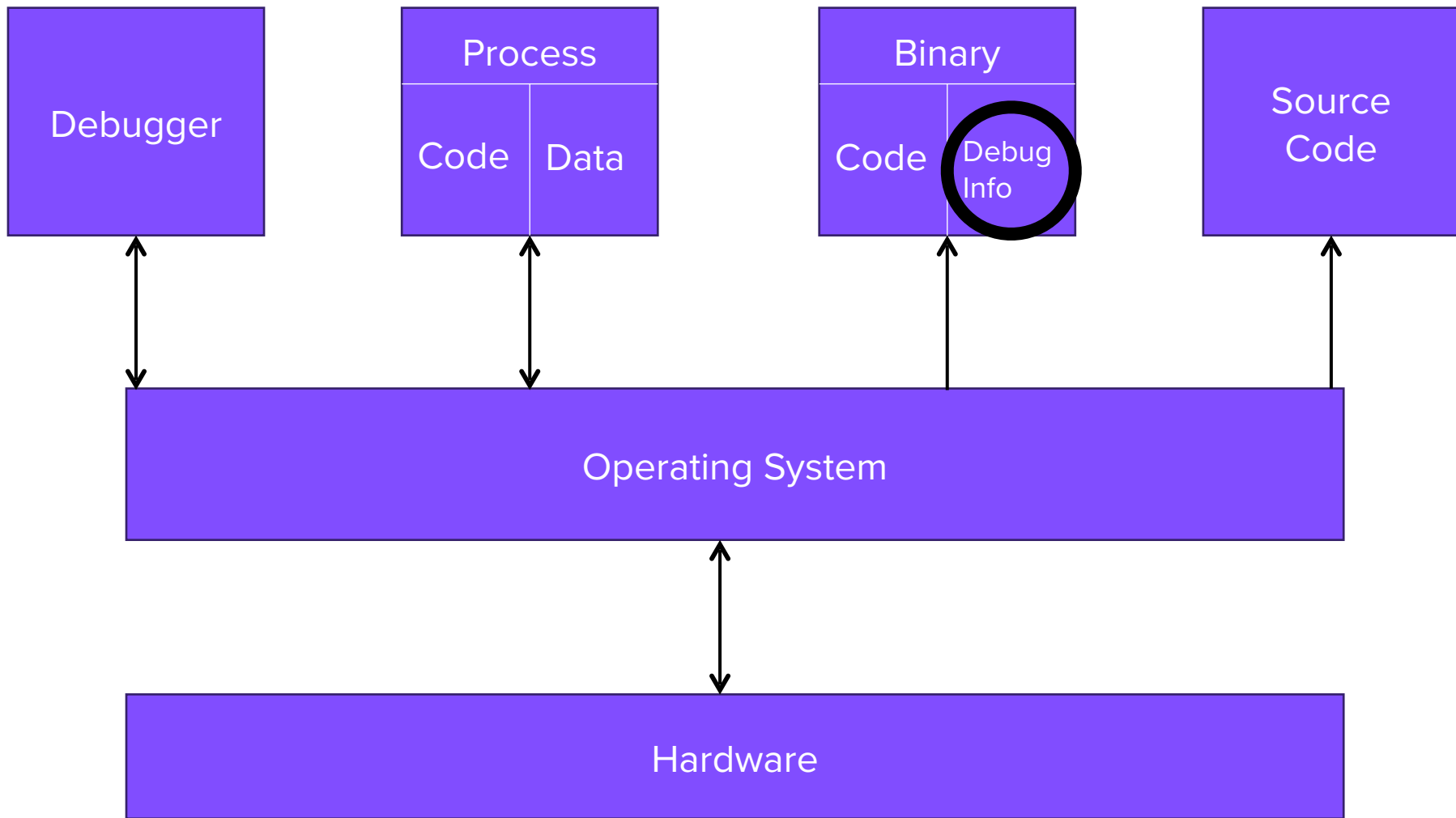
Data generated by the compiler that maps machine code back to the original source code

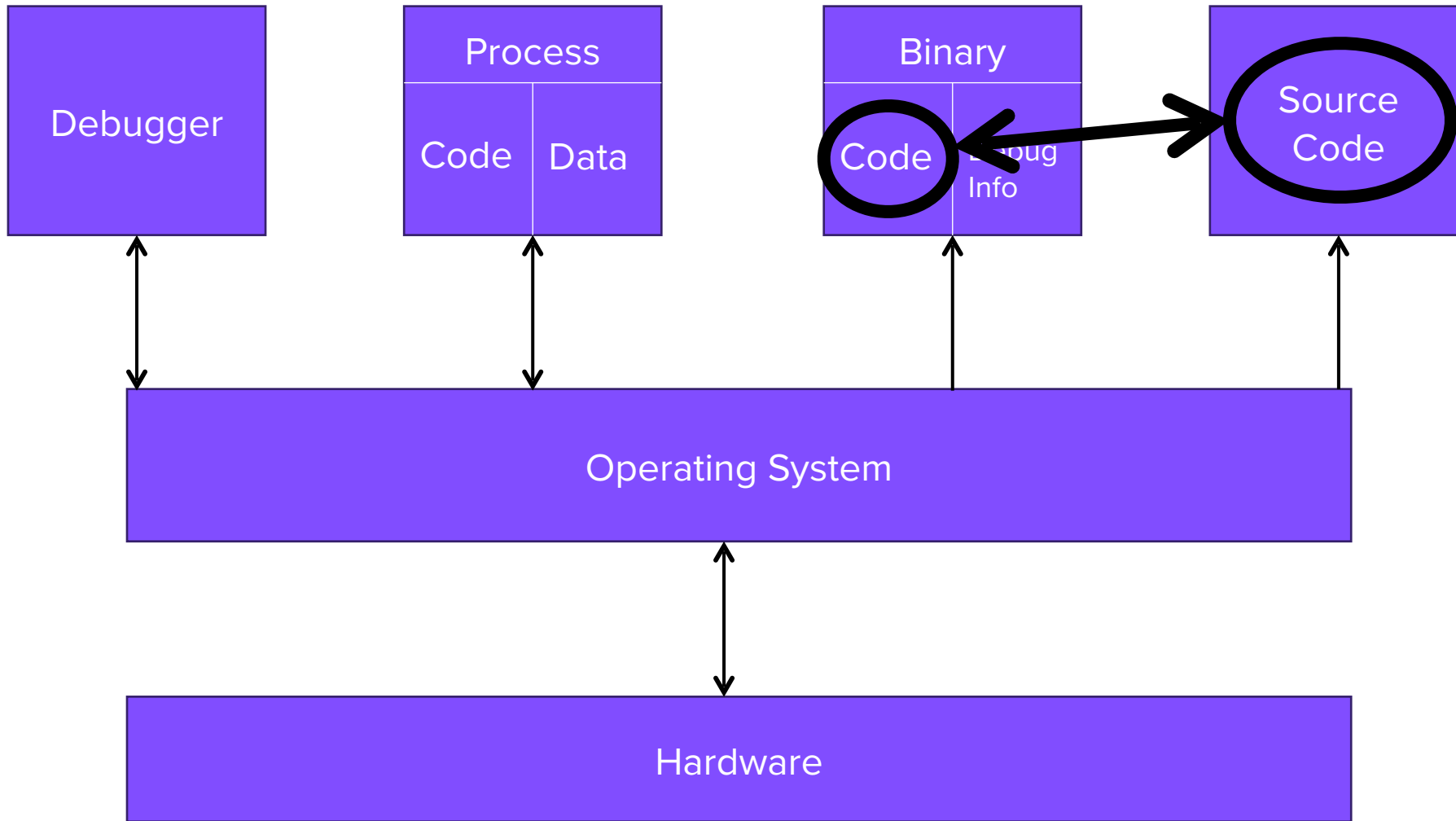
DEBUG INFORMATION

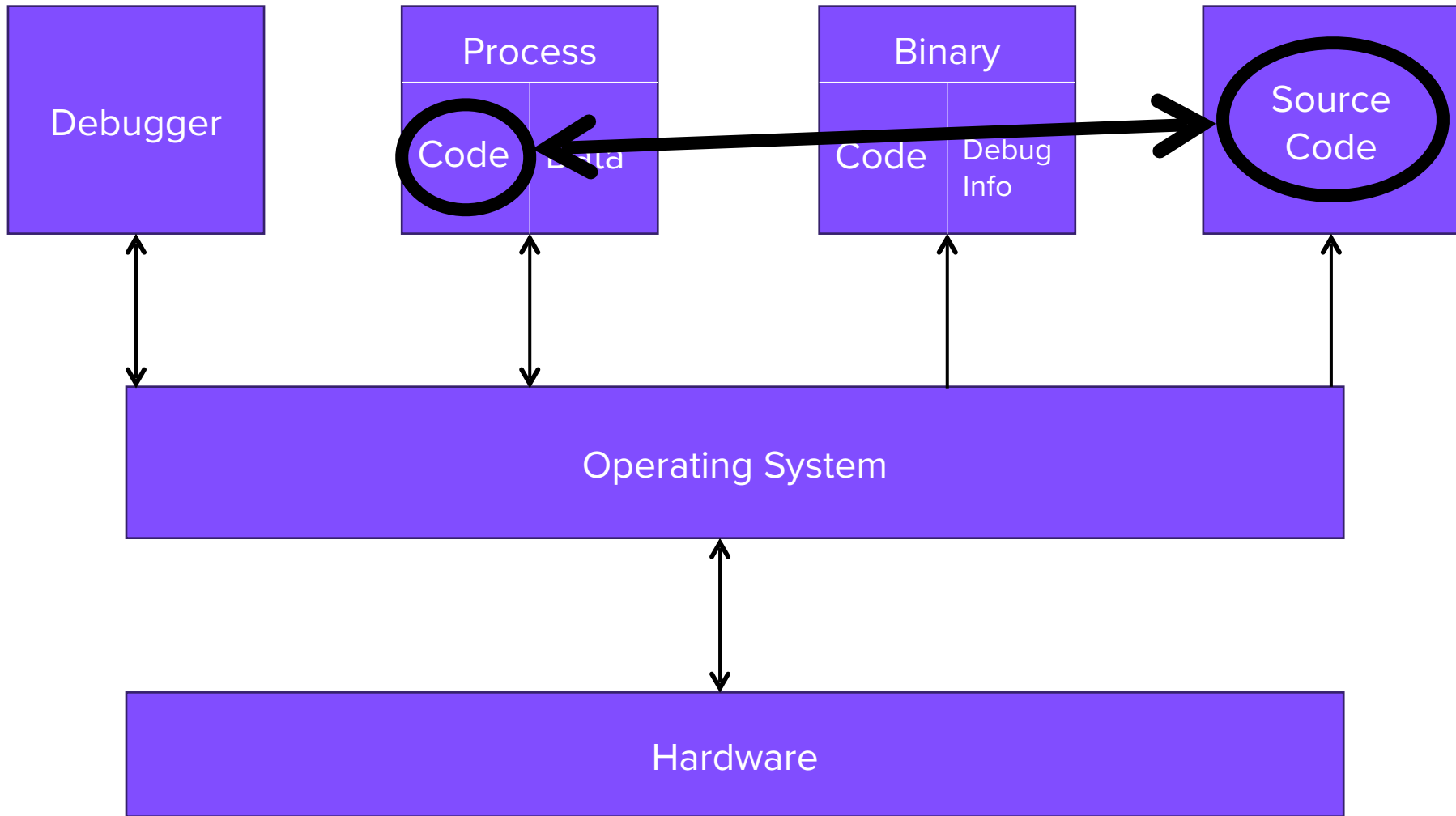
Data generated by the compiler that maps machine code back to the original source code











LINE INFORMATION

Map between lines of source code and addresses of machine instructions

.debug_line:

0x0000186c	[86, 7]	NS	uri: “/home/tartanllama/pet_cat.c”
0x00001878	[86, 23]	NS	
0x00001883	[86, 37]	NS	
0x00001886	[86, 37]	NS	ET
0x00001886	[149, 5]	NS	
0x00001896	[151, 19]	NS	
0x000018a2	[151, 7]	NS	
0x000018a6	[152, 16]	NS	
0x000018ab	[153, 5]	NS	

.debug_line:

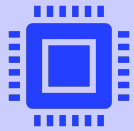
0x0000186c	[86, 7]	NS	uri: “/home/tartanllama/pet_cat.c”
0x00001878	[86, 23]	NS	
0x00001883	[86, 37]	NS	
0x00001886	[86, 37]	NS	ET
0x00001886	[149, 5]	NS	
0x00001896	[151, 19]	NS	
0x000018a2	[151, 7]	NS	
0x000018a6	[152, 16]	NS	
0x000018ab	[153, 5]	NS	

.debug_line:

0x0000186c	[86, 7]	NS	uri: “/home/tartanllama/pet_cat.c”
0x00001878	[86, 23]	NS	
0x00001883	[86, 37]	NS	
0x00001886	[86, 37]	NS	ET
0x00001886	[149, 5]	NS	
0x00001896	[151, 19]	NS	
0x000018a2	[151, 7]	NS	
0x000018a6	[152, 16]	NS	
0x000018ab	[153, 5]	NS	

.debug_line:

0x0000186c	[86, 7]	NS	uri: “/home/tartanllama/pet_cat.c”
0x00001878	[86, 23]	NS	
0x00001883	[86, 37]	NS	
0x00001886	[86, 37]	NS	ET
<u>0x00001886</u>	[<u>149</u> , 5]	NS	
0x00001896	[151, 19]	NS	
0x000018a2	[151, 7]	NS	
0x000018a6	[152, 16]	NS	
0x000018ab	[153, 5]	NS	



Which machine instruction corresponds to that source line?



How do we make the process stop when that machine instruction is executed?

Hardware Breakpoints

- Write the breakpoint address into a debug register
- Limited by number of debug registers
- Can break on read/write/execute

Software Breakpoints

- Modify running code
- Unlimited
- Can only break on execution

89 c6
48 89 df
e8 7e fe ff ff

mov %eax,%esi
mov %rbx,%rdi
call 1130

89 c6
48 89 df
e8 7e fe ff ff

mov %eax,%esi
mov %rbx,%rdi
call 1130

89 c6
cc 89 df
e8 7e fe ff ff

mov %eax,%esi
int3
call 1130

Software interrupt

- “Interrupts” whatever code is currently running
- Transfers control to an operating system interrupt handler
- Interrupt handler takes action depending on the interrupt type

int3

- Trigger interrupt
- Notify operating system
- Signal process to stop
- Debugger intercepts signal and reports to user



89 c6
cc 89 df
e8 7e fe ff ff

mov %eax,%esi
int3
call 1130

→ 89 c6
cc 89 df
e8 7e fe ff ff

mov %eax,%esi
int3
call 1130



89 c6
cc 89 df 
e8 7e fe ff ff

mov %eax,%esi
int3
call 1130

89 c6
cc → 89 df 
e8 7e fe ff ff

mov %eax,%esi
int3
call 1130



```
89 c6  
48 89 df  
e8 7e fe ff ff
```

```
mov    %eax,%esi  
mov    %rbx,%rdi  
call   1130
```

89 c6
48 89 df
→ e8 7e fe ff ff

mov %eax,%esi
mov %rbx,%rdi
call 1130

89 c6
cc 89 df
→ e8 7e fe ff ff

mov %eax,%esi
int3
call 1130

89 c6
cc 89 df
e8 7e fe ff ff
→ ...

mov %eax,%esi
int3
call 1130

WATCHPOINTS



Hardware Breakpoints

- Write the breakpoint address into a debug register
- Limited by number of debug registers
- Can break on read/write/execute

Software Breakpoints

- Modify running code
- Unlimited
- Can only break on execution

Hardware Breakpoints

- Write the breakpoint address into a debug register
- Limited by number of debug registers
- **Can break on read/write/execute**

Software Breakpoints

- Modify running code
- Unlimited
- Can only break on execution

Address registers DR0-DR3

0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000000

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000000

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000000

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000000

Enable
bits

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000000

(Unused)

Enable
bits

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000000

Size + Condition
bits

(Unused)

Enable
bits

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000000 00000000 00000001

Size + Condition
bits

(Unused)

Enable
bits

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00000011 00000000 00000001

Size + Condition
bits

(Unused)

Enable
bits

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00001111 00000000 00000001

Size + Condition
bits

(Unused)

Enable
bits

Address registers DR0-DR3

0xCA	0xFE	0xCA	0xFE
0xBA	0x5E	0xBA	0x11
0x00	0x00	0x00	0x00
0x00	0x00	0x00	0x00

Control register DR7

00000000 00001111 00000000 00000001

Size + Condition
bits

(Unused)

Enable
bits

Status register DR6

.....1



AGENDA

Debug scenario architecture

Manipulating memory

Breakpoints

Watchpoints

Stepping

Stack unwinding

Reading a variable

Calling a function



AGENDA

Debug scenario architecture

Manipulating memory

Breakpoints

Watchpoints

Stepping

Stack unwinding

Reading a variable

Calling a function

STEPPING



Types of stepping

- Instruction-level step
- Source-level step
 - Step in
 - Step over
 - Step out

Instruction-level step

Instruction-level step

```
ptrace(PTRACE_SINGLESTEP, pid, nullptr, nullptr);
```

Step out

Step out

```
auto addr = process.find_return_address();  
process.set_breakpoint(addr);  
process.continue();
```

Step in

Step in

```
auto start_line = source_line_at(process.rip);
```

Step in

```
auto start_line = source_line_at(process.rip);  
do {  
    process.step_instruction();  
} while (source_line_at(process.rip) == start_line);
```

Step over

Step over

```
auto start_line = source_line_at(process.rip);
do {
```

```
} while (source_line_at(process.rip) == start_line);
```

Step over

```
auto start_line = source_line_at(process.rip);  
do {  
    if (instruction_at(process.rip).is_call) {  
  
    } else {  
  
    }  
} while (source_line_at(process.rip) == start_line);
```

Step over

```
auto start_line = source_line_at(process.rip);
do {
    if (instruction_at(process.rip).is_call) {
        auto addr = instruction_after(process.rip);
        process.set_breakpoint(addr);
        process.continue();
    } else {

    }
} while (source_line_at(process.rip) == start_line);
```

Step over

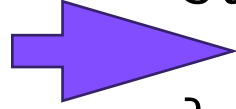
```
auto start_line = source_line_at(process.rip);
do {
    if (instruction_at(process.rip).is_call) {
        auto addr = instruction_after(process.rip);
        process.set_breakpoint(addr);
        process.continue();
    } else {
        process.step_instruction();
    }
} while (source_line_at(process.rip) == start_line);
```

STACK UNWINDING

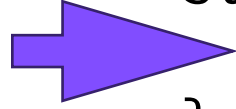
```
std::optional<animal> cat::find_preay() {  
    // ...  
}  
  
void cat::nap() {  
    // ...  
}
```

```
std::optional<animal> cat::find_preay() {  
    // ...  
}  
  
void cat::nap() {  
    // ...  
}  
  
void cat::hunt() {  
    std::optional<animal> prey = find_preay();  
    if (prey) chase(*prey);  
    nap();  
}
```

```
std::optional<animal> cat::find_preay() {  
    // ...  
}  
  
void cat::nap() {  
    // ...  
}  
  
void cat::hunt() {  
    std::optional<animal> prey = find_preay();  
    if (prey) chase(*prey);  
    nap();  
}  
  
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```



```
std::optional<animal> cat::find_preay() {  
    // ...  
}  
  
void cat::nap() {  
    // ...  
}  
  
void cat::hunt() {  
    std::optional<animal> prey = find_preay();  
    if (prey) chase(*prey);  
    nap();  
}  
  
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```



```
std::optional<animal> cat::find_prej() {  
    // ...  
}
```

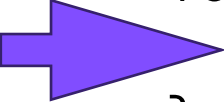
```
void cat::nap() {  
    // ...  
}
```

```
void cat::hunt() {  
    std::optional<animal> prey = find_prej();  
    if (prey) chase(*prey);  
    nap();  
}
```

```
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```

```
* frame #0: 0x00000000100000410 a.out`cat::find_prej()  
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28  
frame #2: 0x0000000010000053c a.out`main + 40  
frame #3: 0x0000000019f776b98 dyld`start + 6076
```

```
std::optional<animal> cat::find_preay() {  
    // ...  
}
```



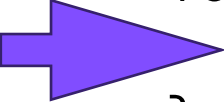
```
void cat::nap() {  
    // ...  
}
```

```
* frame #0: 0x00000000100000464 a.out`cat::nap()  
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28  
frame #2: 0x0000000010000053c a.out`main + 40  
frame #3: 0x0000000019f776b98 dyld`start + 6076
```

```
void cat::hunt() {  
    std::optional<animal> prey = find_preay();  
    if (prey) chase(*prey);  
    nap();  
}
```

```
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```

```
std::optional<animal> cat::find_prej() {  
    // ...  
}
```



```
void cat::nap() {  
    // ...  
}
```

frame #0: 0x00000000100000464 a.out`cat::nap()
* frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076

```
void cat::hunt() {  
    std::optional<animal> prey = find_prej();  
    if (prej) chase(*prej);  
    nap();  
}
```

```
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```

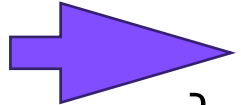
```
std::optional<animal> cat::find_prej() {  
    // ...  
}
```

```
void cat::nap() {  
    // ...  
}
```

```
void cat::hunt() {  
    std::optional<animal> prey = find_prej();  
    if (prey) chase(*prey);  
    nap();  
}
```

```
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```

frame #0: 0x00000000100000464 a.out`cat::nap()
* frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



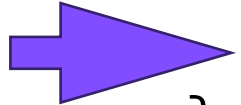
```
std::optional<animal> cat::find_prej() {  
    // ...  
}
```

```
void cat::nap() {  
    // ...  
}
```

```
void cat::hunt() {  
    std::optional<animal> prej = find_prej();  
    if (prej) chase(*prej);  
    nap();  
}
```

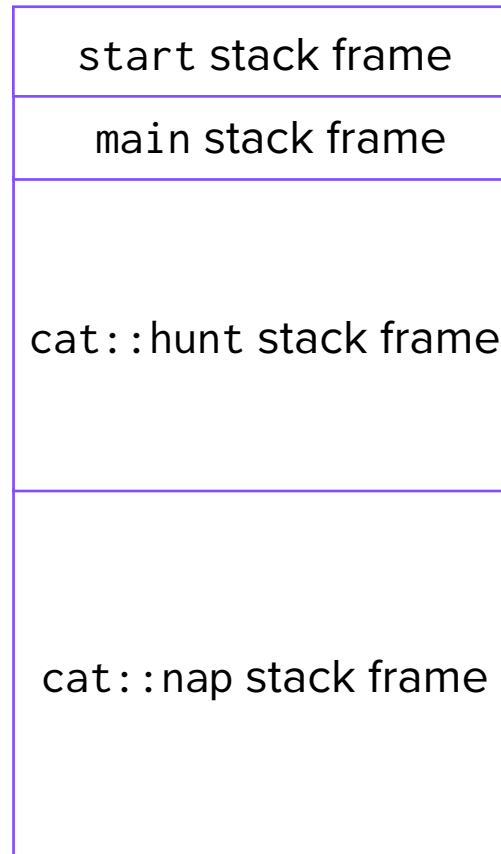
```
int main() {  
    cat poptart("poptart");  
    poptart.hunt();  
}
```

frame #0: 0x00000000100000464 a.out`cat::nap()
* frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076

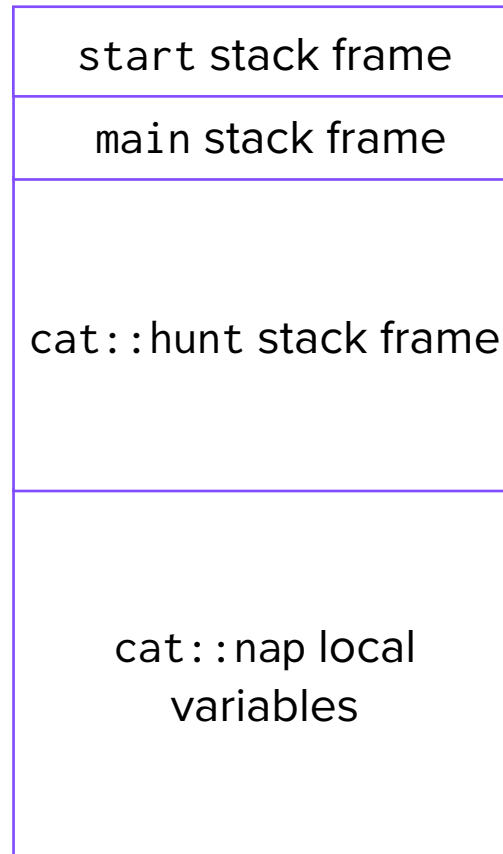


```
* frame #0: 0x00000000100000464 a.out`cat::nap()  
  frame #1: 0x00000000100000490 a.out`cat::hunt() + 28  
  frame #2: 0x0000000010000053c a.out`main + 40  
  frame #3: 0x0000000019f776b98 dyld`start + 6076
```

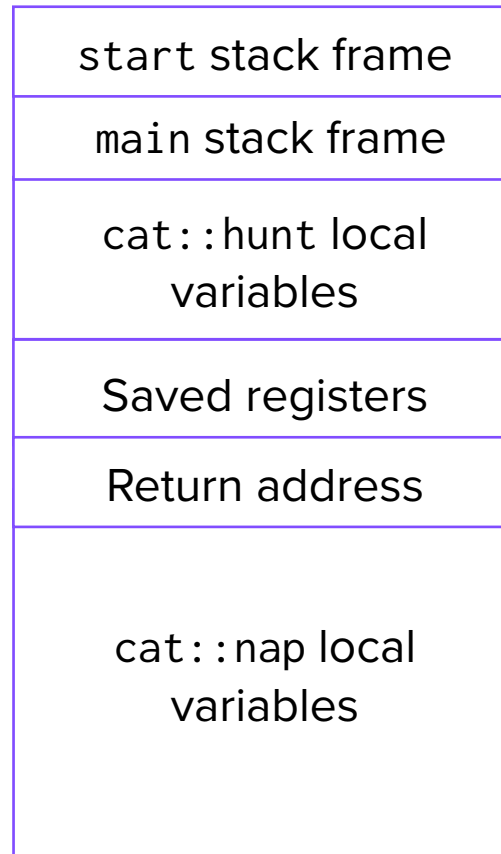
* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



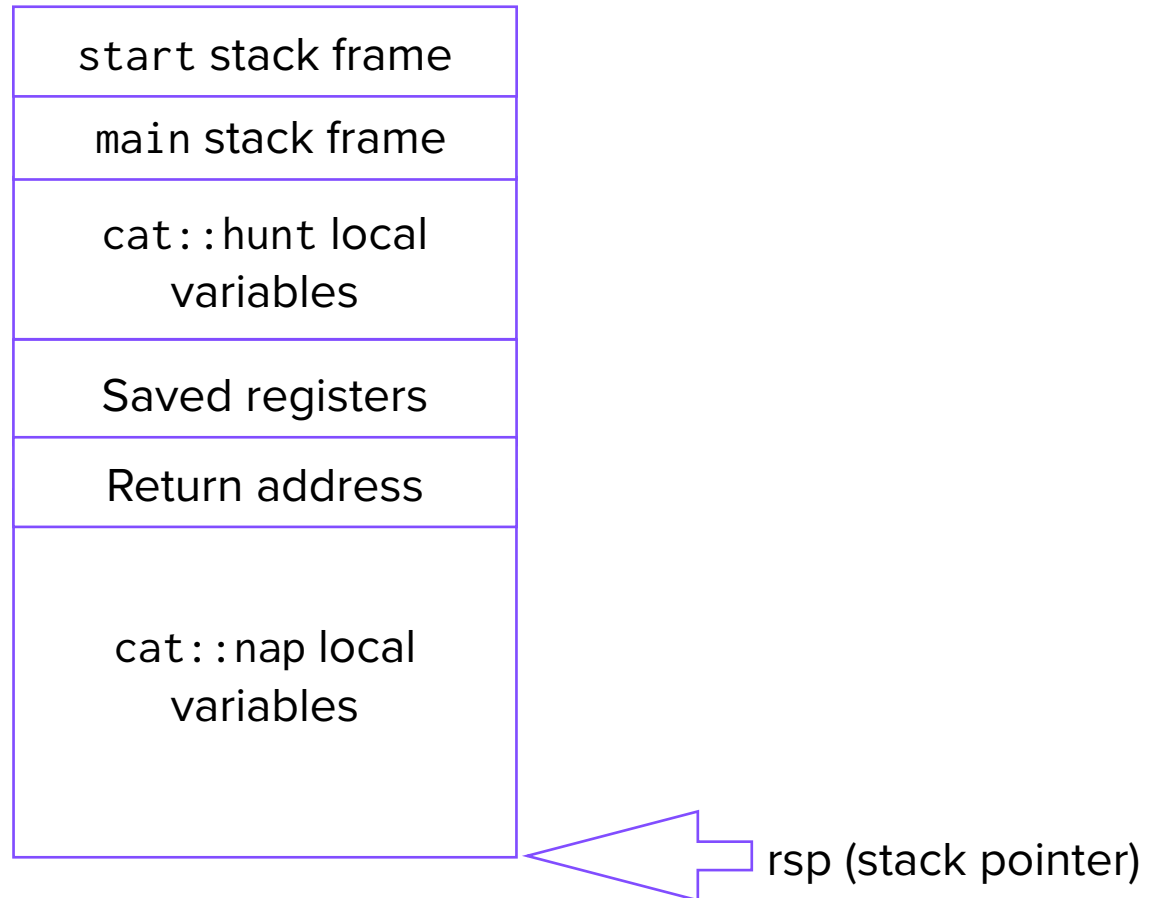
* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



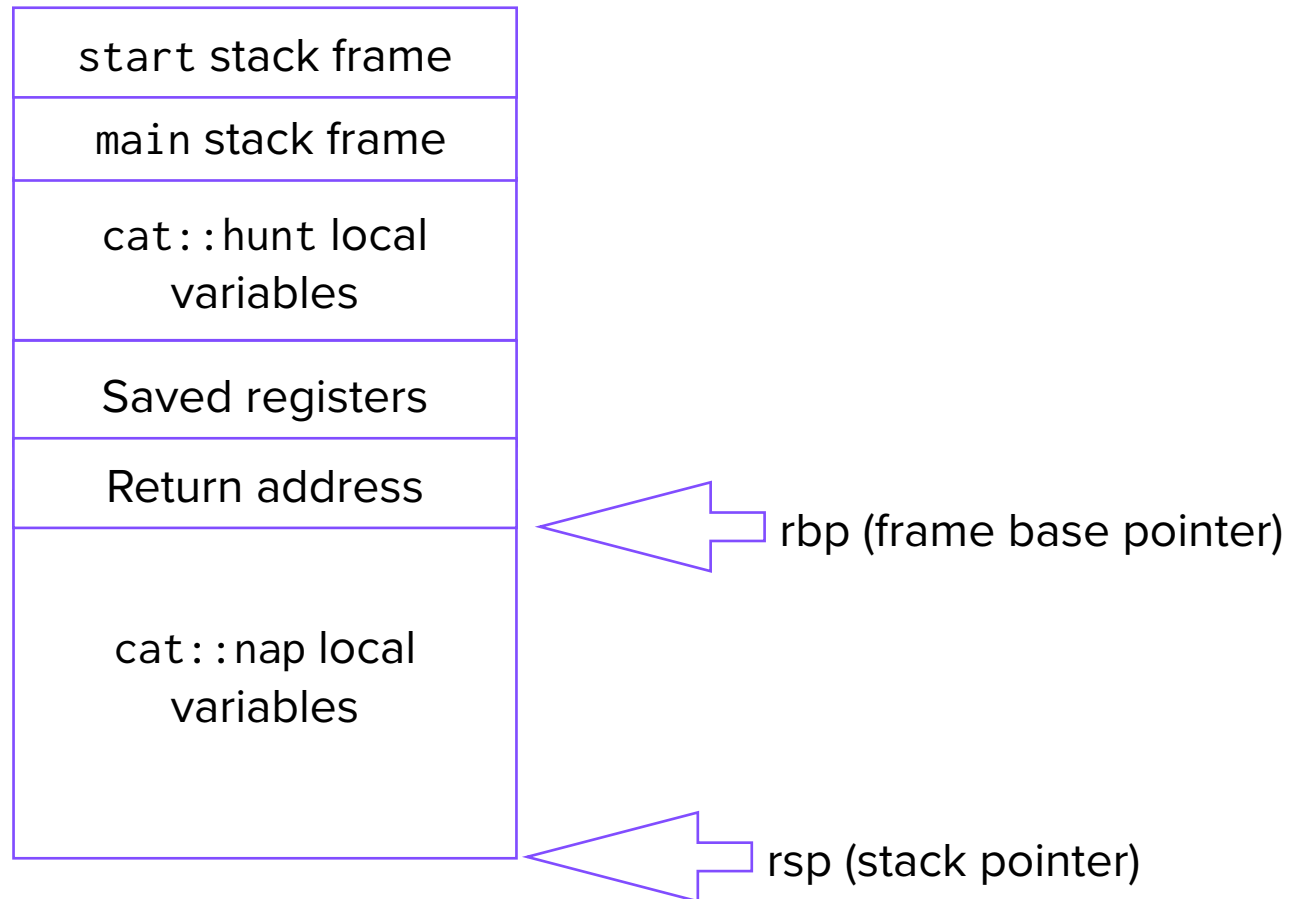
* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



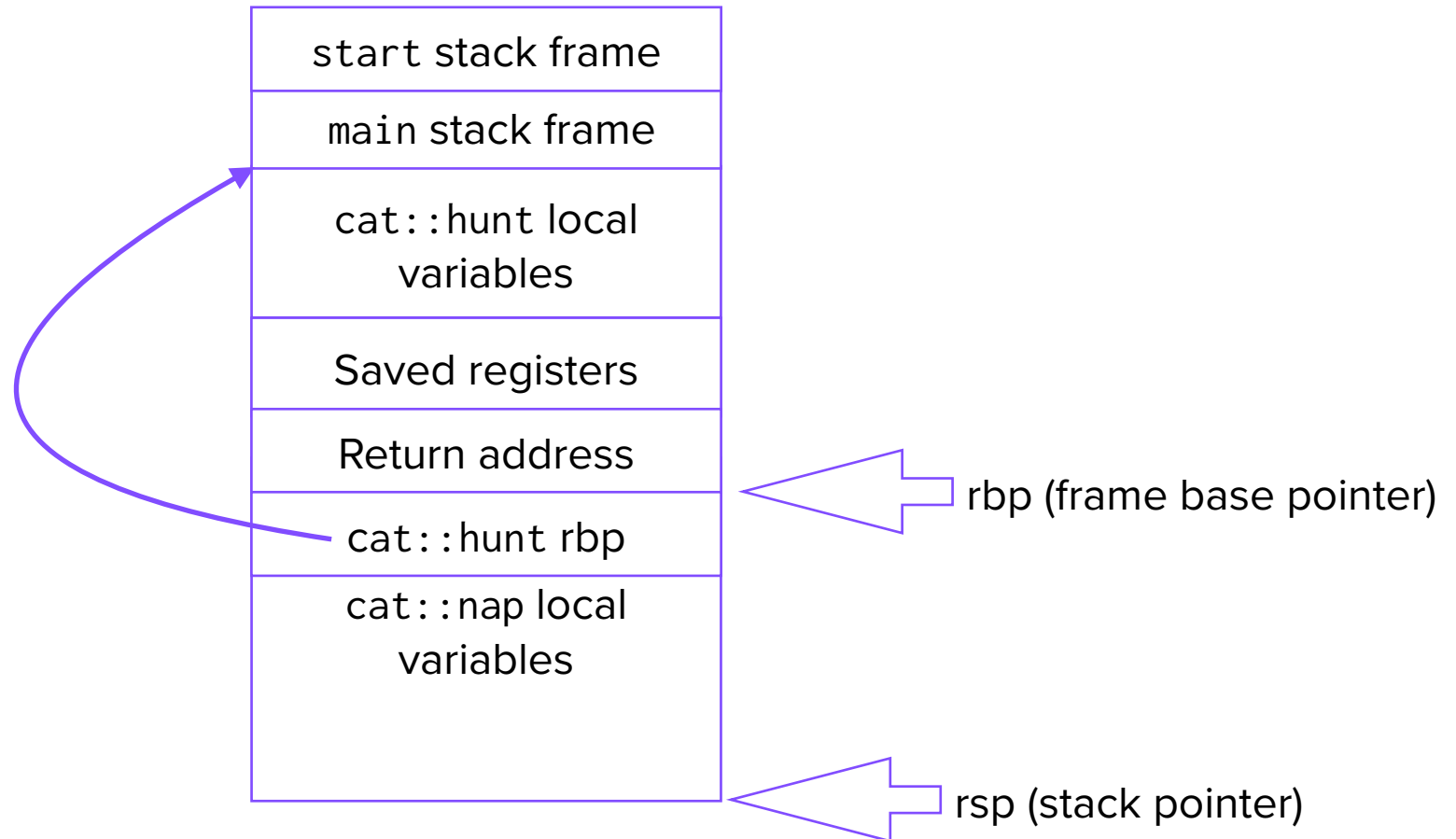
* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



Call Frame Information

Call Frame Information

Current RIP	Canonical Frame Address (CFA)	Old RIP	Old RBP
0x401150			
0x401151			
0x401154			
0x40115a			

Call Frame Information

Current RIP	Canonical Frame Address (CFA)	Old RIP	Old RBP
0x401150	RSP+8	[CFA-8]	(Unchanged)
0x401151			
0x401154			
0x40115a			

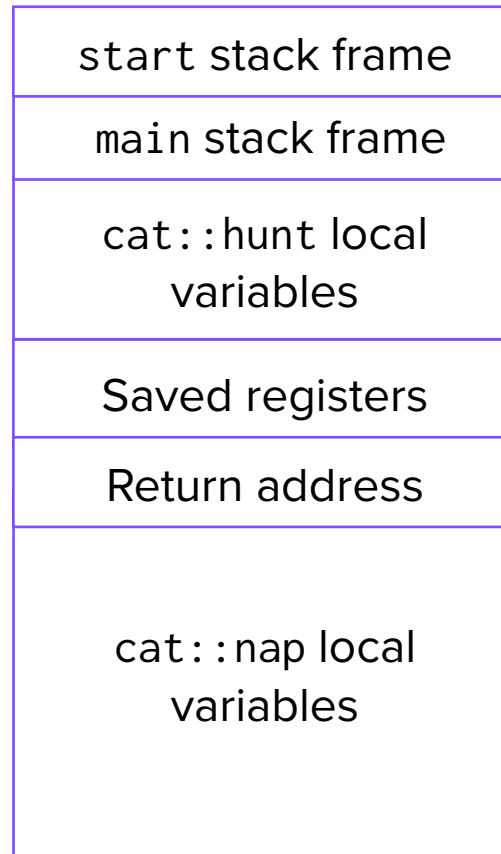
Call Frame Information

Current RIP	Canonical Frame Address (CFA)	Old RIP	Old RBP
0x401150	RSP+8	[CFA-8]	(Unchanged)
0x401151	RSP+16	[CFA-8]	[CFA-16]
0x401154			
0x40115a			

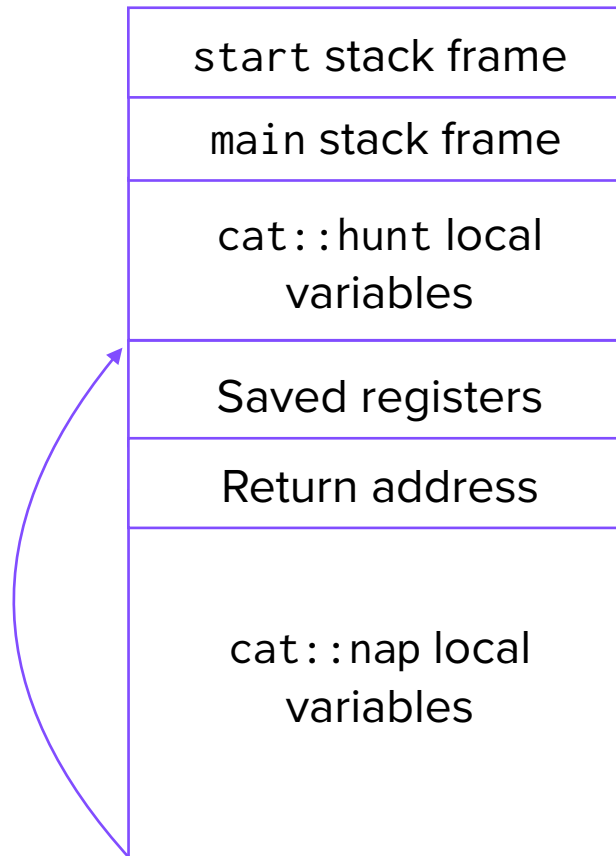
Call Frame Information

Current RIP	Canonical Frame Address (CFA)	Old RIP	Old RBP
0x401150	RSP+8	[CFA-8]	(Unchanged)
0x401151	RSP+16	[CFA-8]	[CFA-16]
0x401154	RBP+16	[CFA-8]	[CFA-16]
0x40115a	RSP+8	[CFA-8]	[CFA-16]

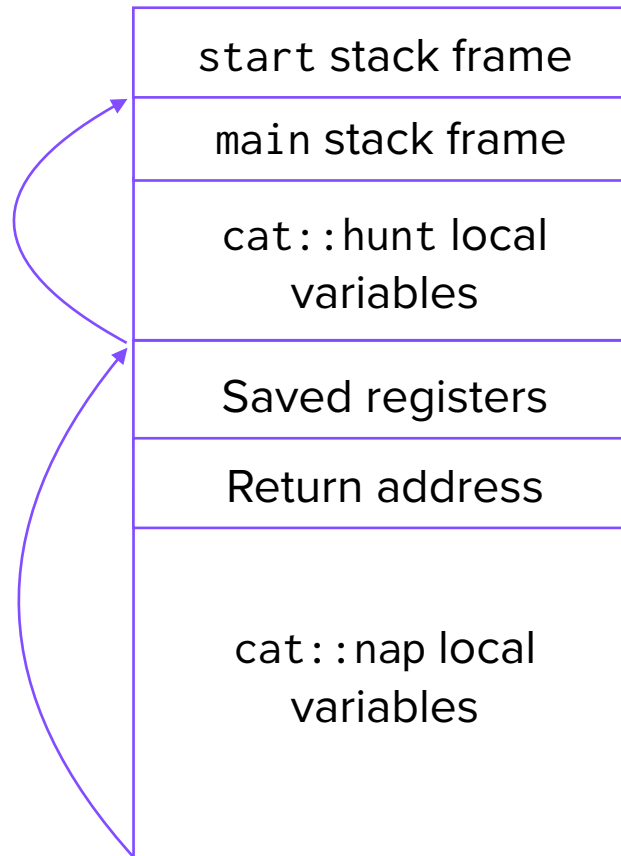
* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



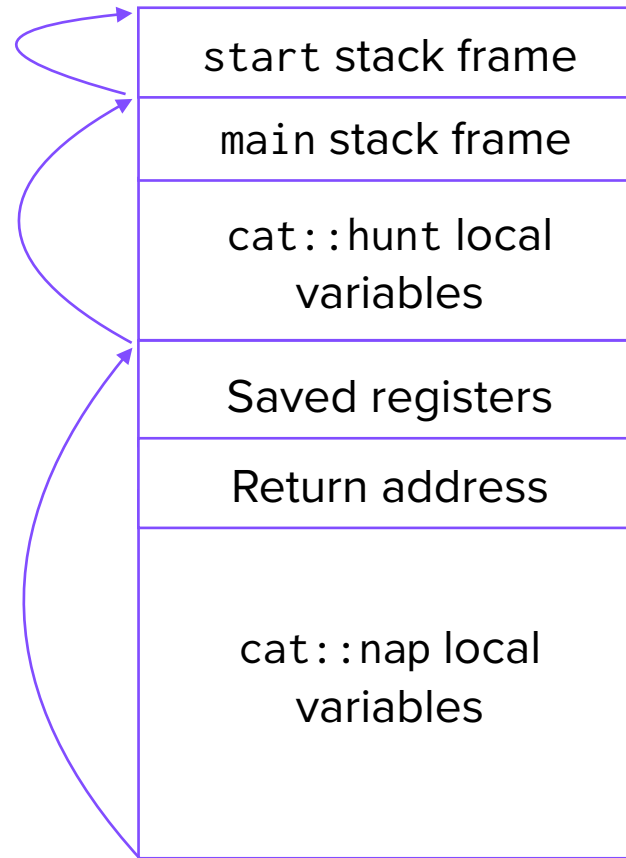
* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076



* frame #0: 0x00000000100000464 a.out`cat::nap()
frame #1: 0x00000000100000490 a.out`cat::hunt() + 28
frame #2: 0x0000000010000053c a.out`main + 40
frame #3: 0x0000000019f776b98 dyld`start + 6076





READING A VARIABLE

Visualise the variable milkshake



```
struct cat {  
    const char* name;  
    int floofiness;  
    int age;  
};  
  
cat milkshake = {  
    "milkshake", 10, 3  
};
```

55 55 60 04

00 00 00 0a

00 00 55 55

00 00 00 03

55 55 60 04

00 00 00 0a

00 00 55 55

00 00 00 03

55 55 60 04
00 00 00 0a

00 00 55 55
00 00 00 03

55 55 60 04

00 00 00 0a

00 00 55 55

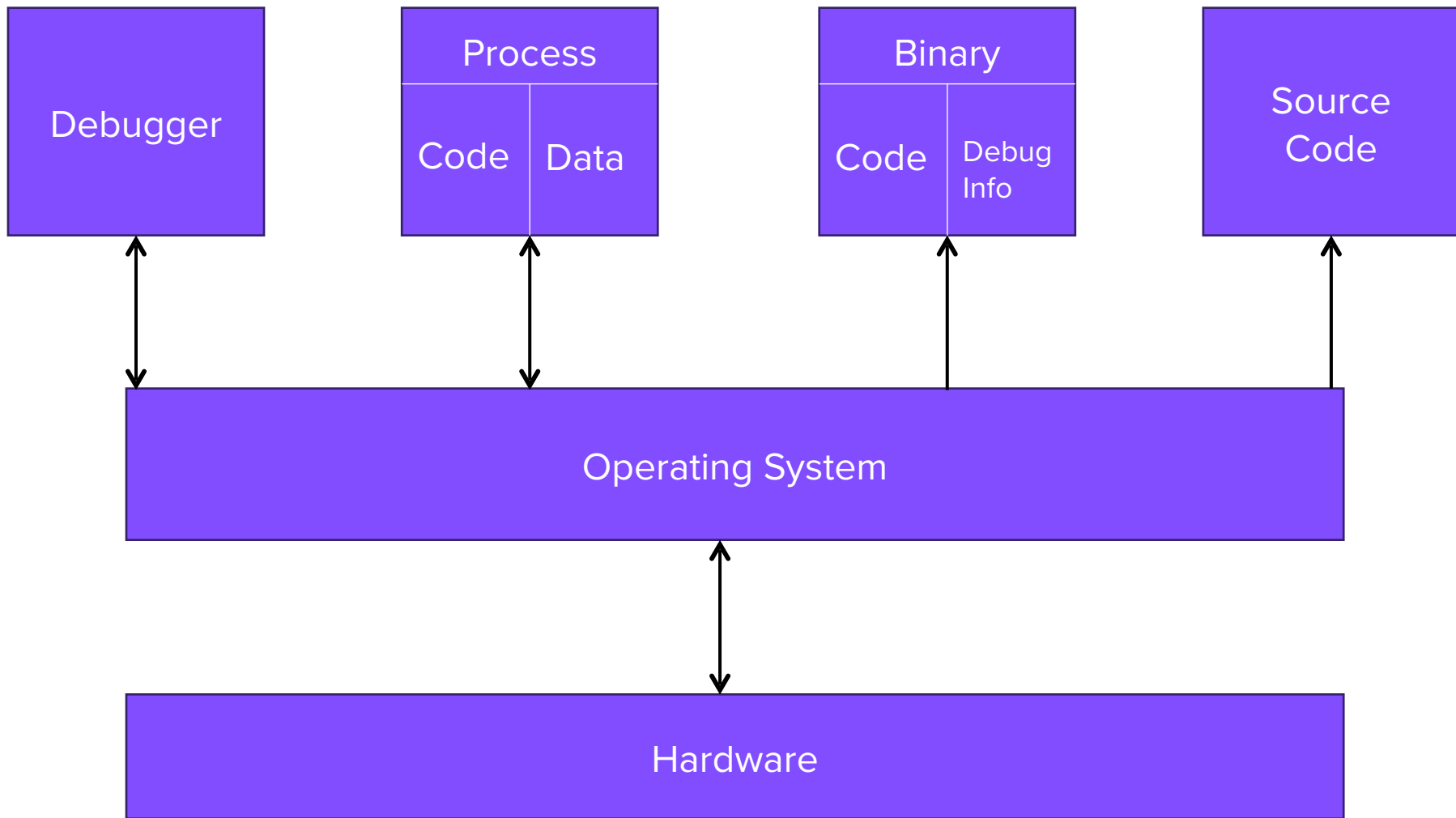
00 00 00 03

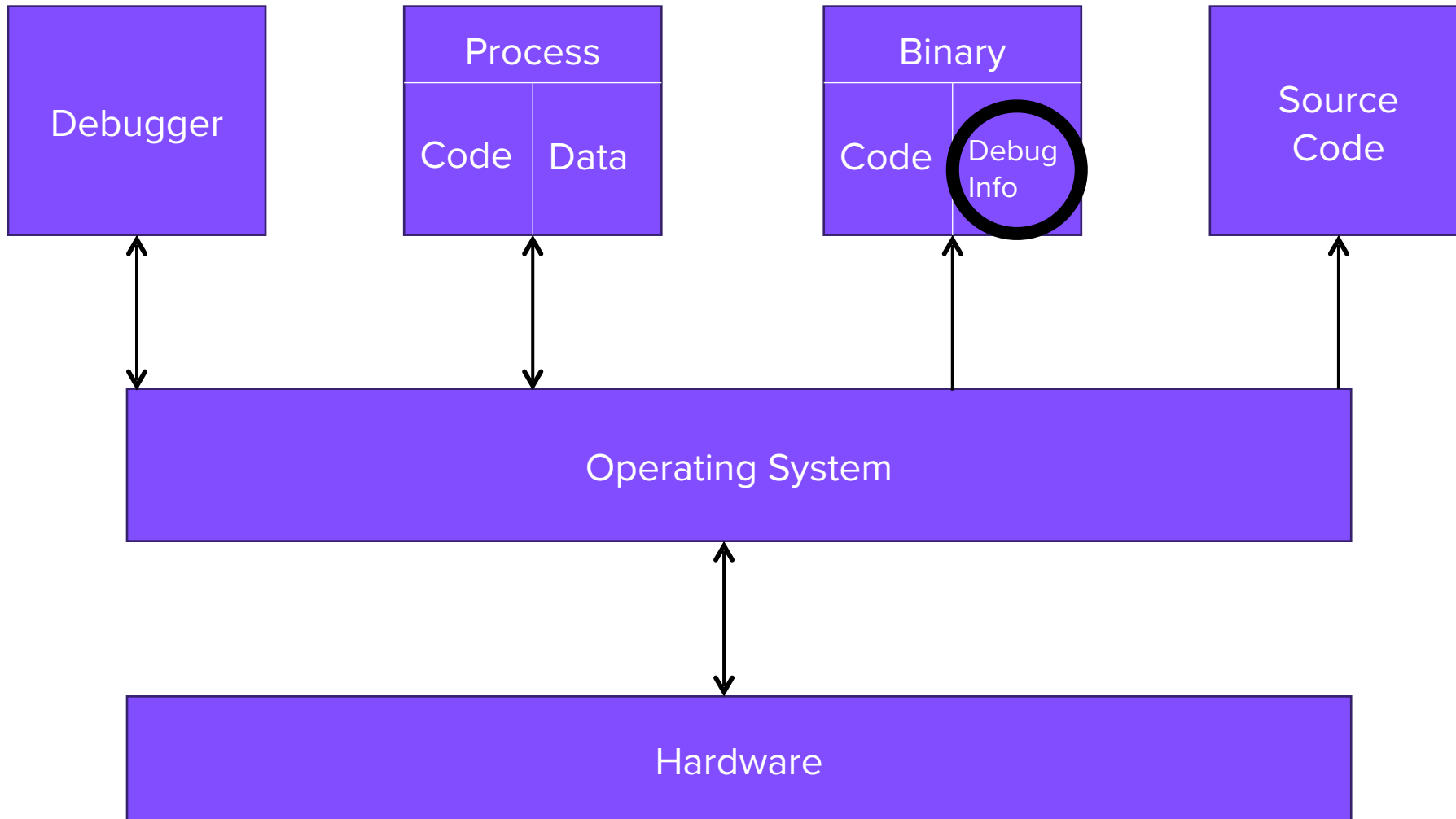


Where does the object live?



How should we interpret the bytes there?





```

DW_TAG_structure_type
    DW_AT_name          cat
    DW_AT_byte_size     0x00000010
    DW_TAG_member
        DW_AT_name      name
        DW_AT_type       <0x00000066>
        DW_AT_data_member_location 0
    DW_TAG_member
        DW_AT_name      floofiness
        DW_AT_type       <0x00000078>
        DW_AT_data_member_location 8
    DW_TAG_member
        DW_AT_name      age
        DW_AT_type       <0x00000078>
        DW_AT_data_member_location 12

```

DW_TAG_variable	
DW_AT_name	milkshake
DW_AT_type	<0x00000061>
DW_AT_location	
DW_OP_addr	0x00003df0

55 55 60 04

00 00 00 0a

00 00 55 55

00 00 00 03

```
milkshake = {  
    name = "milkshake",  
    floofiness = 10,  
    age = 3  
};
```



AGENDA

Debug scenario architecture

Manipulating memory

Breakpoints

Watchpoints

Stepping

Stack unwinding

Reading a variable

Calling a function



AGENDA

Debug scenario architecture

Manipulating memory

Breakpoints

Watchpoints

Stepping

Stack unwinding

Reading a variable

Calling a function



CALLING A FUNCTION

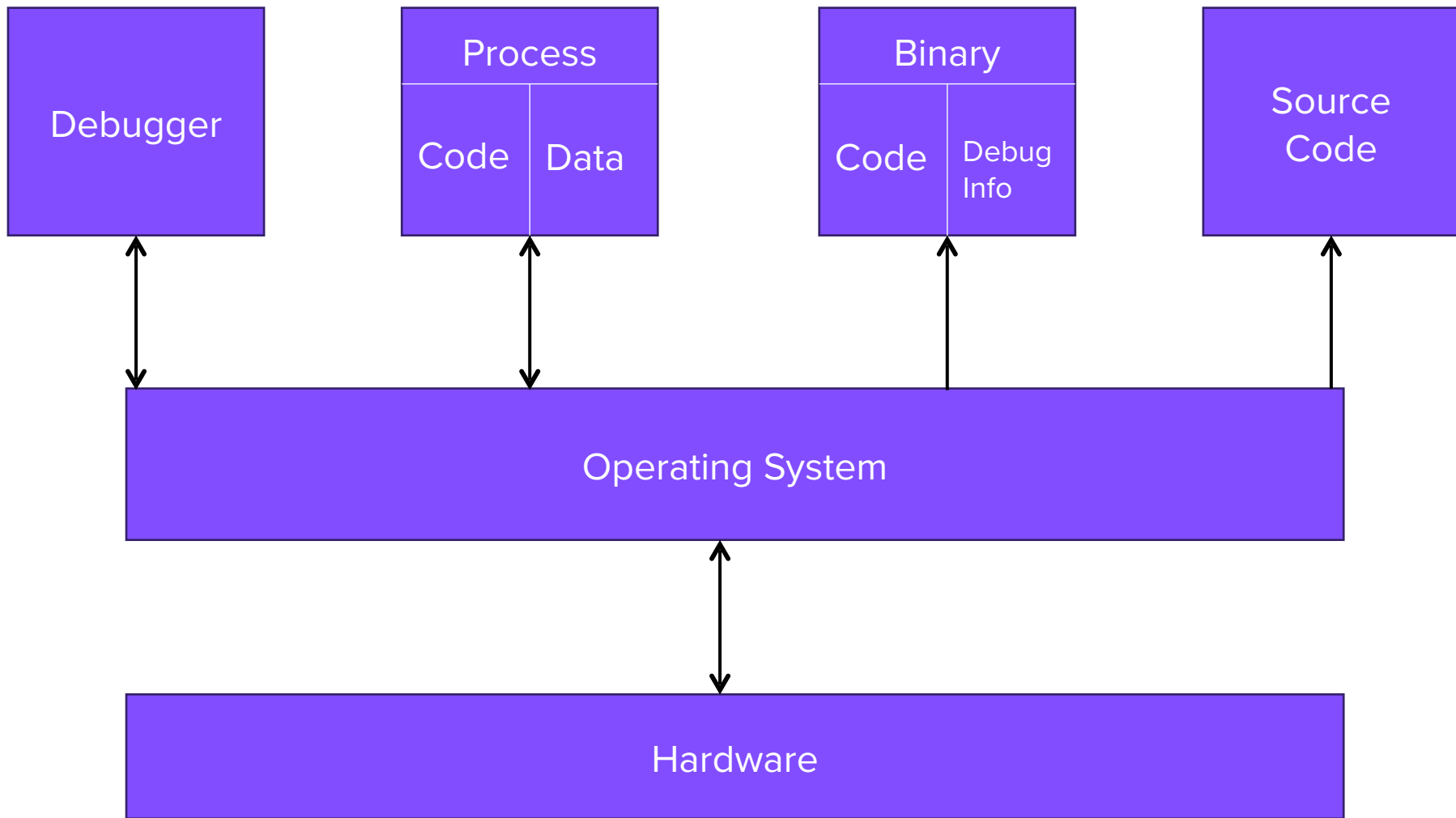
Call the function `cat::feed(int)`

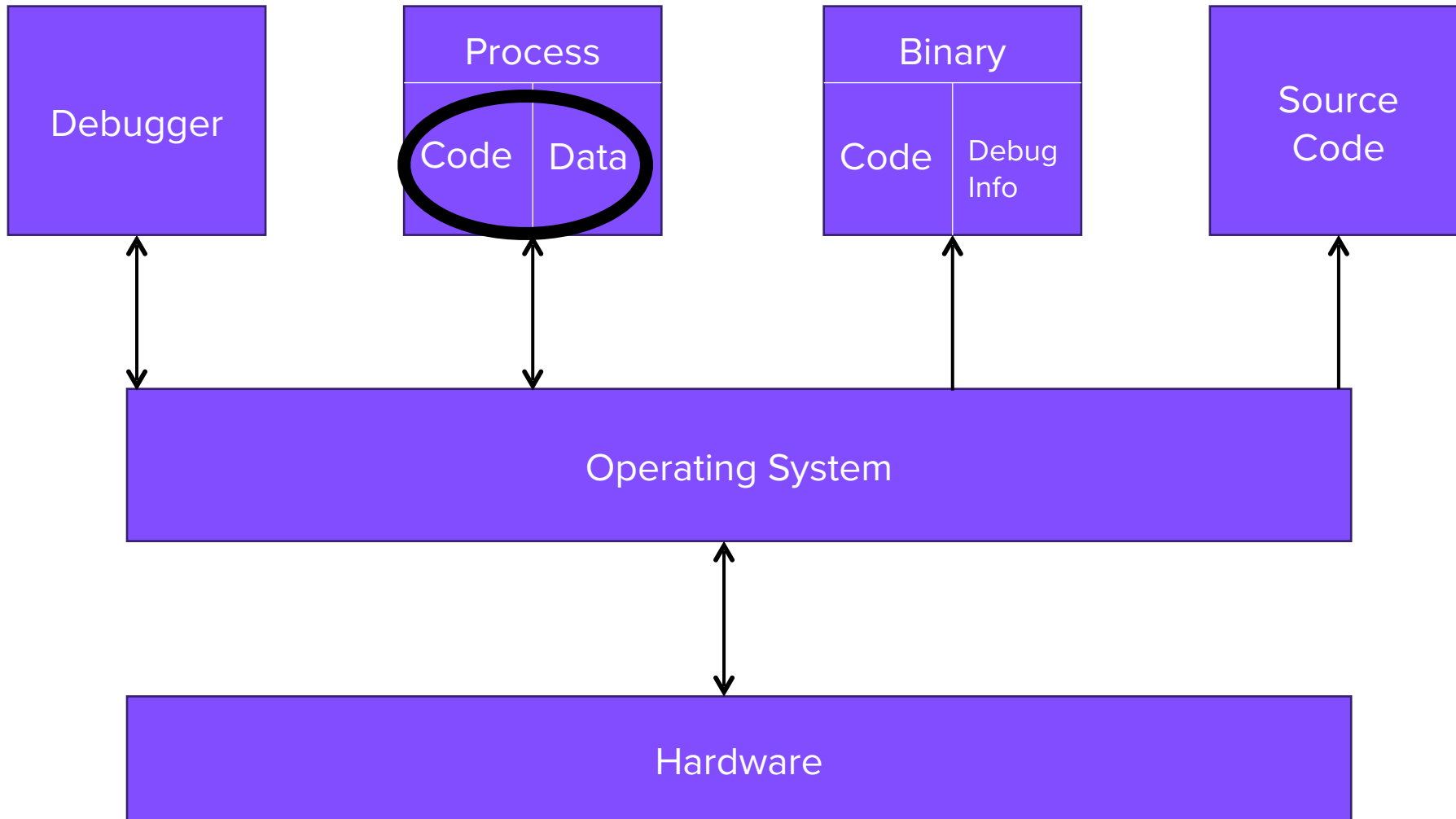


```
struct cat {  
    int feed(int n_treats);  
};
```

```
struct cat {  
    int feed(int n_treats);  
};
```

```
gdb) poptart.feed(12)  
4
```





Step 1: Write a C++ compiler

System V Application Binary Interface
AMD64 Architecture Processor Supplement
(With LP64 and ILP32 Programming Models)
Version 1.0

Edited by
H.J. Lu¹, Michael Matz², Milind Girkar³, Jan Hubička⁴,
Andreas Jaeger⁵, Mark Mitchell⁶



June 3, 2024



```
DW_TAG_structure_type  
    DW_AT_name ("cat")  
    DW_AT_byte_size (0x01)
```

```
DW_TAG_subprogram  
    DW_AT_linkage_name ("_ZN3cat4feedEi")  
    DW_AT_type (0x00000040 "int")  
    DW_AT_name ("feed")
```

```
DW_TAG_formal_parameter  
    DW_AT_type (0x00000044 "cat *")  
    DW_AT_artificial (true)
```

```
DW_TAG_formal_parameter  
    DW_AT_type (0x00000040 "int")
```

```
DW_TAG_structure_type
  DW_AT_name ("cat")
  DW_AT_byte_size (0x01)
```

```
DW_TAG_subprogram
  DW_AT_linkage_name ("_ZN3cat4feedEi")
  DW_AT_type (0x00000040 "int")
  DW_AT_name ("feed")
```

```
DW_TAG_formal_parameter
  DW_AT_type (0x00000044 "cat *")
  DW_AT_artificial (true)
```

```
DW_TAG_formal_parameter
  DW_AT_type (0x00000040 "int")
```

Itanium C++ ABI

Contents

- [Acknowledgements](#)
- [Chapter 1: Introduction](#)
 - [1.1 Definitions](#)
 - [1.2 Limits](#)
 - [1.3 Namespace and Header](#)
 - [1.4 Scope of This ABI](#)
 - [1.5 Base Documents](#)
- [Chapter 2: Data Layout](#)
 - [2.1 General](#)
 - [2.2 POD Data Types](#)
 - [2.3 Member Pointers](#)
 - [2.4 Non-POD Class Types](#)
 - [2.5 Virtual Table Layout](#)
 - [2.6 Virtual Tables During Object Construction](#)
 - [2.7 Array Operator `new` Cookies](#)
 - [2.8 Initialization Guard Variables](#)
 - [2.9 Run-Time Type Information \(RTTI\)](#)
- [Chapter 3: Code Emission and APIs](#)
 - [3.1 Functions](#)
 - [3.2 Virtual Calls](#)
 - [3.3 Construction and Destruction APIs](#)
 - [3.4 Demangler API](#)
- [Chapter 4: Exception Handling](#)
- [Chapter 5: Linkage and Object Files](#)
 - [5.1 External Names \(a.k.a. Mangling\)](#)
 - [5.2 Vague Linkage](#)
 - [5.3 Unwind Table Location](#)
- [Appendix R: Revision History](#)

```
struct cat {  
    int feed(int n_treats);  
};
```

```
struct cat {  
    int feed(int n_treats);  
};
```

```
int _ZN3cat4feedEi(cat*, int);
```

```
int _ZN3cat4feedEi(cat*, int);
```

```
gdb) poptart.feed(12)
```

1. Parse the expression
2. Look up feed function
3. Find address of poptart variable
4. Set up arguments
5. Set the program counter to the feed function
6. Print return value
7. Restore old program state

```
int _ZN3cat4feedEi(cat*, int);
```

```
gdb) poptart.feed(12)
```

RAX	RDI	RSI

```
int _ZN3cat4feedEi(cat*, int);
```

```
gdb) popstart.feed(12)
```

RAX	RDI	RSI
	0XBA5EBA11	12

```
int _ZN3cat4feedEi(cat*, int);
```

```
gdb) popstart.feed(12)
```

RAX	RDI	RSI
4	0XBABACACA	12

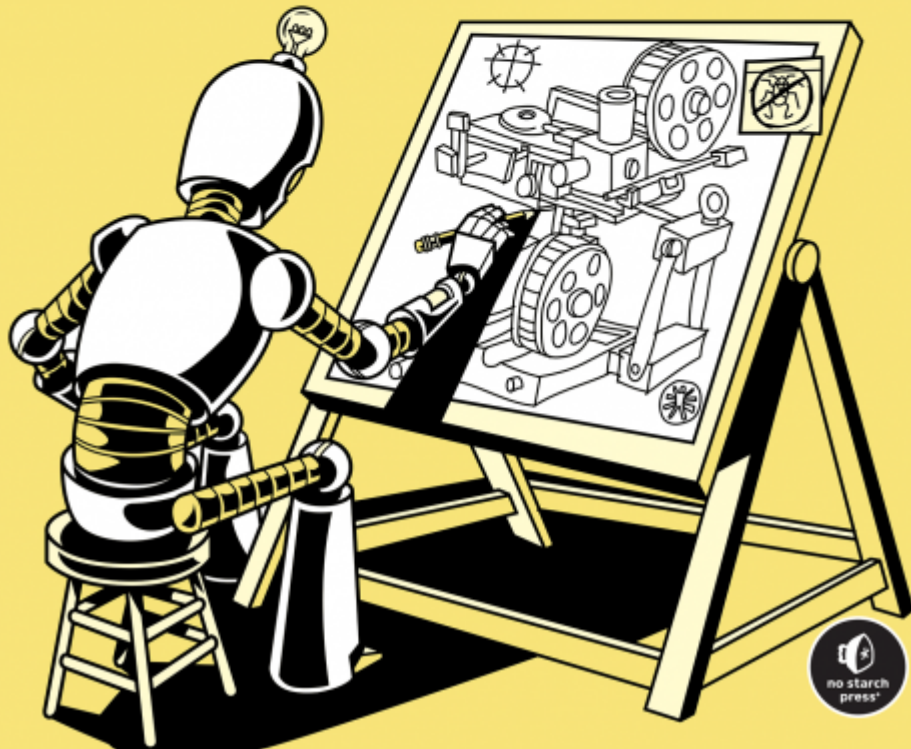
```
int _ZN3cat4feedEi(cat*, int);
```

gdb) popstart.feed(12)
4

RAX	RDI	RSI
4	0XBABACACA	12

BUILDING A DEBUGGER

SY BRAND



<https://nostarch.com/building-a-debugger>

HOW DEBUGGERS WORK

Sy Brand
they/them

Staff Software Engineer, Web Assembly @ Fastly
@TartanLlama

+

○

●



QUESTIONS?

ASK ME IN THE APP

RATE THIS SESSION

YES, YOU'LL GET A REWARD